Building data course — Week 3 — fault detection in SQL and pandas

sparql-playground / lessons

Generated for print and offline reading

Week 3 — fault detection in SQL and pandas

Five daily lessons, all run from Python. Each computation is done twice, as a SQLite query string (stdlib `sqlite3`) and in pandas, and `lab.assert_parity` checks they agree. Run a day with `uv run python lessons/course.py run N` (it uses a temporary copy of `var/timeseries.sqlite`; build that with `uv run bldgts bootstrap`). `assert` lines state the answers you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 11 — Long to wide, and into SI units: the `history` table every Open-FDD rule reads
- Day 12 — Indexes and `EXPLAIN QUERY PLAN`
- Day 13 — Window functions: LAG, rolling averages, streaks
- Day 14 — Write a fault rule (FC1, duct static too low) twice, and check it against Open-FDD
- Day 15 — What production does differently (Timescale), and the UUID bridge to RDF

Day 11 — Long to wide, and into SI units: the `history` table every Open-FDD rule reads

Goal: pivot the readings into one row per AHU per timestamp, one column per role, in SQL *and* in pandas, and convert everything to SI on the way.

Read

Day 8 needed six joins to put three points side by side. Fault rules want *many* points side by side, so do the pivot once:

```
long (samples + lab_point)                          wide (lab_history)
equipment ts     role          value                equipment ts
fan_status duct_static duct_static_sp ...
AHU_1     10:05  fan-status    1          ──▶      AHU_1     10:05  1
335.6       349.3
AHU_1     10:05  duct-static   1.31
AHU_1     10:05  duct-static-sp 1.40
```

The data is in **long** form (one row per reading) and rules want **wide** form (one row per moment). Each side has its own idiom:

Step	SQL	pandas
attach the role to each reading	`JOIN lab_point p ON p.timeseries_id = s.timeseries_id`	`samples.merge(points, on="timeseries_id")`
one row per (equipment, ts)	`GROUP BY p.equipment_id, s.ts`	`pivot(index=["equipment_id", "ts"], ...)`
one column per role	`MAX(CASE WHEN role = 'x' THEN value END) AS col`	`columns="openfdd_role"`
convert units	an expression inside the `MAX(CASE ...)`	arithmetic on the pivoted column

The SQL trick is **conditional aggregation** again (Day 9): each group has exactly one reading per role, so `MAX` simply picks it, and the other rows give NULL, which `MAX` ignores. pandas' `pivot` does the same job directly, and it is stricter: if a group had *two* readings for one role it raises an error instead of quietly picking one (`pivot_table(aggfunc="max")` is the forgiving version, the same as SQL's `MAX`).

Open-FDD's SQL rules (see `sql/fdd/postgres/rules/*.sql`) all read a relation called `history` with columns named after roles: `fan_status`, `fan_cmd`, `duct_static`, `duct_static_sp`, `sat`, `mat`, `rat`, `oat`… The column names below follow that **input contract**, so a rule written against it doesn't care which building it runs on.

Why the history is SI

The historian logged what the building controller showed: °F, inches of water column, percent 0–100. In weeks 1–2 you explored the data as logged. **From today on, the data every rule reads is SI**: temperatures in °C, pressure in pascals, commands and positions as fractions 0–1. Open-FDD's fault math is SI-only for three reasons:

- **One rule, any building.** A threshold like "25 Pa below setpoint" or "2.8 °C from setpoint" means the same thing on a US site logging °F and a European one logging °C, as long as the conversion happens once, at the door.

- **No unit logic inside rules.** A rule that guessed units would need an `if` on every comparison. Convert once in the `history` build and every rule stays a plain comparison.
- **Fractions compose.** A damper at `0.35` times a flow is a flow. A damper at `35` times a flow needs a `/100` somewhere, and sooner or later someone forgets it.

Quantity	As logged	In `history`	Conversion
temperatures (`sat`, `sat_sp`, `mat`, `rat`, `oat`)	°F	°C	`(v - 32) * 5 / 9`
pressures (`duct_static`, `duct_static_sp`)	inH2O	Pa	`v * 249.08891`
commands and positions (`fan_cmd`, `oa_damper`, `clg_valve`)	% 0–100	fraction 0–1	`v / 100`
statuses (`fan_status`, `occupied`)	0 / 1	0 / 1	none

Note for later: the `- 32` only applies to a **temperature**. A temperature *difference* of 5 °F is 5 × 5/9 = 2.78 °C, with no 32 in it. Day 15 needs that.

You'll make the history a **table**, not a view: the pivot reads 924k rows and takes a few seconds. As a table it's built once and every rule after that is fast (the Day 10 summary-table idea).

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_points(conn)  # lab_point: timeseries_id -> equipment,
         openfdd_role (Day 8)
```

In SQL. The `MAX(CASE ...)` is written out in full, with the conversion for each column inside it.

```
HISTORY_SQL = """
SELECT p.equipment_id, s.ts,
  MAX(CASE WHEN p.openfdd_role = 'fan-status' THEN s.value END) AS
        fan_status,
  MAX(CASE WHEN p.openfdd_role = 'fan-cmd' THEN s.value / 100.0 END) AS
        fan_cmd,
  MAX(CASE WHEN p.openfdd_role = 'duct-static-pressure' THEN s.value *
        249.08891 END) AS duct_static,
  MAX(CASE WHEN p.openfdd_role = 'duct-static-pressure-sp' THEN s.value
        * 249.08891 END) AS duct_static_sp,
  MAX(CASE WHEN p.openfdd_role = 'discharge-air-temp' THEN (s.value -
        32.0) * 5.0 / 9.0 END) AS sat,
  MAX(CASE WHEN p.openfdd_role = 'discharge-air-temp-sp' THEN (s.value
        - 32.0) * 5.0 / 9.0 END) AS sat_sp,
  MAX(CASE WHEN p.openfdd_role = 'mixed-air-temp' THEN (s.value - 32.0)
        * 5.0 / 9.0 END) AS mat,
  MAX(CASE WHEN p.openfdd_role = 'return-air-temp' THEN (s.value -
        32.0) * 5.0 / 9.0 END) AS rat,
  MAX(CASE WHEN p.openfdd_role = 'outside-air-temp' THEN (s.value -
        32.0) * 5.0 / 9.0 END) AS oat,
  MAX(CASE WHEN p.openfdd_role = 'outside-air-damper' THEN s.value /
        100.0 END) AS oa_damper,
  MAX(CASE WHEN p.openfdd_role = 'cooling-valve' THEN s.value / 100.0
        END) AS clg_valve,
  MAX(CASE WHEN p.openfdd_role = 'occupied' THEN s.value END) AS
        occupied
FROM samples s
JOIN lab_point p ON p.timeseries_id = s.timeseries_id
WHERE p.openfdd_role IS NOT NULL
GROUP BY p.equipment_id, s.ts
"""
conn.execute("DROP TABLE IF EXISTS lab_history")
conn.execute("CREATE TABLE lab_history AS " + HISTORY_SQL)
conn.execute(
    "CREATE INDEX lab_history_eq_ts ON lab_history (equipment_id, ts)"
)  # Day 12 explains why
conn.commit()

history_sql = lab.query(conn, "SELECT * FROM lab_history ORDER BY
        equipment_id, ts")
print(len(history_sql), "rows")
assert len(history_sql) == 71072  # 2 AHUs x 35,536 timestamps
```

In pandas. Read the readings and the point list as two DataFrames, merge them, then pivot. Reading all 4.2 million samples takes a moment; the merge keeps only the 924k that have a role.

```
samples = lab.query(conn, "SELECT timeseries_id, ts, value FROM
        samples")
points = lab.query(
    conn,
    "SELECT timeseries_id, equipment_id, openfdd_role FROM lab_point
        WHERE openfdd_role IS NOT NULL",
)
long = samples.merge(
    points, on="timeseries_id", how="inner"
)  # inner join: unmapped series drop out
print(len(samples), "samples ->", len(long), "readings with a role")

# one row per (equipment, ts), one column per role. pivot() raises if
        any cell had two readings.
wide = long.pivot(index=["equipment_id", "ts"], columns="openfdd_role",
        values="value")
print(wide.shape)
```

Now the SI conversion, one explicit line per column, using the converters in `lab` (`f_to_c`, `inh2o_to_pa`) and plain arithmetic:

```
history_pd = pd.DataFrame(index=wide.index)
history_pd["fan_status"] = wide["fan-status"]
history_pd["fan_cmd"] = wide["fan-cmd"] / 100.0
history_pd["duct_static"] = lab.inh2o_to_pa(wide["duct-static-
        pressure"])
history_pd["duct_static_sp"] = lab.inh2o_to_pa(wide["duct-static-
        pressure-sp"])
history_pd["sat"] = lab.f_to_c(wide["discharge-air-temp"])
history_pd["sat_sp"] = lab.f_to_c(wide["discharge-air-temp-sp"])
history_pd["mat"] = lab.f_to_c(wide["mixed-air-temp"])
history_pd["rat"] = lab.f_to_c(wide["return-air-temp"])
history_pd["oat"] = lab.f_to_c(wide["outside-air-temp"])
history_pd["oa_damper"] = wide["outside-air-damper"] / 100.0
history_pd["clg_valve"] = wide["cooling-valve"] / 100.0
history_pd["occupied"] = wide["occupied"]
history_pd = history_pd.reset_index().sort_values(["equipment_id",
        "ts"]).reset_index(drop=True)

lab.assert_parity(history_pd, history_sql, keys=["equipment_id", "ts"],
        label="wide history")
```

Every cell of the SQL table equals the pandas one (the `MAX(CASE ...)` and the `pivot` are two spellings of the same reshape). One more check, so later days can trust the helper: `lab.ensure_history` builds this same table from `lab.HISTORY_COLUMNS` (role plus conversion per column), and `lab.history_sql()` is the SQL it runs. Same data?

```
helper = lab.query(conn, lab.history_sql())
lab.assert_parity(
    history_sql, helper, keys=["equipment_id", "ts"], label="lesson
        table == lab helper"
)
lab.ensure_history(conn)  # no-op now: lab_history exists, so later
        days reuse it
```

Look at the result, and at what SI did to the numbers. Day 8's six-join query is now one line of logic. The old `1.347 inH2O` average duct static has become pascals:

```
running = history_sql[history_sql.fan_status == 1]
by_ahu = running.groupby("equipment_id").agg(
    avg_static_pa=("duct_static", "mean"),
    avg_setpoint_pa=("duct_static_sp", "mean"),
    rows=("ts", "size"),
)
print(by_ahu.round(1))

AVG_SQL = """
SELECT equipment_id, avg(duct_static) AS avg_static_pa,
         avg(duct_static_sp) AS avg_setpoint_pa,
       count(*) AS rows
FROM lab_history
WHERE fan_status = 1
GROUP BY equipment_id
"""
lab.assert_parity(by_ahu, lab.query(conn, AVG_SQL), label="average
         static while running")
a1 = by_ahu.loc["AHU_1"]
assert (
    round(a1.avg_static_pa, 1) == 335.6
    and round(a1.avg_setpoint_pa, 1) == 349.3
    and a1.rows == 18893
)
```

AHU_1 holds 335.6 Pa while running against a 349.3 Pa setpoint (1.347 and 1.402 inH2O, the same numbers in other units). That 14 Pa shortfall is invisible in a quick glance and is exactly what Day 14's rule goes looking for.

Try it

1. Data quality: how many rows have the fan running but a discharge, mixed or return air temperature that was logged as exactly **0 °F** (a dead sensor)? Do it in SQL and in pandas.
2. What's missing from this pivot if a role has **no** point on some AHU? (Think about what the column would hold.)

Answers

1. The units changed under this question. In the raw database the dropout test was `sat = 0`. In `lab_history` a 0 °F reading is **-17.78 °C**, so `sat = 0` now means *0 °C* (32 °F), which is a perfectly normal temperature and matches nothing here. The test would silently find zero dropouts. The new test converts the dropout value too, and compares with a small tolerance because these are floating-point numbers:

```
ZERO_F_IN_C = lab.f_to_c(0.0)  # -17.777...
DROPOUT_SQL = """
SELECT count(*) AS dropout_rows FROM lab_history
WHERE fan_status = 1
  AND (abs(sat - :z) < 1e-6 OR abs(mat - :z) < 1e-6 OR abs(rat - :z) <
        1e-6)
"""
dropout_sql = lab.query(conn, DROPOUT_SQL, {"z": ZERO_F_IN_C})

r = history_pd[history_pd.fan_status == 1]
bad = (
    np.isclose(r.sat, ZERO_F_IN_C) | np.isclose(r.mat, ZERO_F_IN_C) |
         np.isclose(r.rat, ZERO_F_IN_C)
)
dropout_pd = pd.DataFrame({"dropout_rows": [int(bad.sum())]})

lab.assert_parity(dropout_pd, dropout_sql, label="0 F dropouts while
         running")
naive = ((r.sat == 0) | (r.mat == 0) | (r.rat == 0)).sum()  # the old
         test, now meaningless
print("dropouts:", dropout_pd.iloc[0, 0], "| old 'sat = 0' test
         finds:", naive)
assert dropout_pd.iloc[0, 0] == 14 and naive == 0
```

Fourteen dropout rows while running: rare, but each one could trip a rule ("discharge air 50 °F below setpoint!"). Production rules filter or flag them. Better still, filter dropouts **before** converting, on the raw value, where "exactly 0" is unambiguous.

2. The column would be NULL (pandas: NaN) for that AHU. That's correct: "not measured" is not 0. A rule that needs that role must then **skip** the AHU and say so, rather than invent data. Open-FDD's contract calls that "skipped / not applicable". Both sides do it the same way. Pretend AHU_2 has no outside-air-temp point:

```
SKIP = "NOT (p.equipment_id = 'AHU_2' AND p.openfdd_role = 'outside-
        air-temp')"
no_oat_sql = lab.history_sql().replace(
    "WHERE p.openfdd_role IS NOT NULL", "WHERE p.openfdd_role IS NOT
        NULL AND " + SKIP
)
sql_missing = lab.query(
    conn,
    f"SELECT equipment_id, count(*) AS rows, count(oat) AS oat_rows
        FROM ({no_oat_sql}) GROUP BY 1",
)

long2 = long[~((long.equipment_id == "AHU_2") & (long.openfdd_role ==
        "outside-air-temp"))]
wide2 = long2.pivot(index=["equipment_id", "ts"],
        columns="openfdd_role", values="value")
wide2 = wide2.reindex(
    columns=wide.columns
)  # a missing role is a column of NaN, not a missing column
oat = wide2["outside-air-temp"]
pd_missing = oat.groupby("equipment_id").agg(rows="size",
        oat_rows="count").reset_index()

lab.assert_parity(pd_missing, sql_missing, label="role missing on one
        AHU")
print(sql_missing.to_string(index=False))
assert sql_missing.set_index("equipment_id").loc["AHU_2", "oat_rows"]
        == 0
```

AHU_2 still has all 35,536 rows, but 0 outside-air-temperature values. `count(col)` and pandas' `count()` both ignore NULL/NaN, which is how a rule can tell "no data" from "zero".

Why it matters

Which UUID becomes `duct_static` for which AHU is a **metadata** question. Here `lab_point.openfdd_role` answered it. In Unit 4 a SPARQL query will: "for each AHU, the point with role duct-static-pressure and its `ref:hasTimeseriesId`". That's the model-driven way to build this exact table for any building. And the unit conversion belongs in that same build step: convert at the door, once, and nothing downstream has to ask.

Day 12 — Indexes and `EXPLAIN QUERY PLAN`

Goal: see how SQLite finds rows, time the same query with and without an index, and see what pandas does instead (a scan, or a sorted index).

Read

A table without a useful index is a list: to find rows, SQLite **scans** all of them, O(n). An **index** is a B-tree sorted by some columns. Finding a key or a key range is O(log n), then it walks the matching rows in order.

`samples` is `WITHOUT ROWID` with primary key `(timeseries_id, ts)`, so the table **is** a B-tree sorted by series, then time. That makes some questions cheap and others not:

Query filters on	Can use the key?
`timeseries_id = ?`	yes, it's the first key column
`timeseries_id = ? AND ts BETWEEN ...`	yes, both columns, range on the second
`ts BETWEEN ...` alone	**no**: time is only sorted *within* each series, so it must scan
`date(ts, 'unixepoch') = ...`	the function hides `ts` from the index

Prefix `EXPLAIN QUERY PLAN` to any query and SQLite tells you its plan instead of running it. Look for `SEARCH ... USING PRIMARY KEY` / `USING INDEX` (good) vs `SCAN table` (reads everything).

The tradeoff: every index speeds up some reads, but costs disk space and slows every insert. Historians index exactly what they're queried by (series + time) and little else.

The same idea in pandas

A DataFrame has no query planner. You choose the algorithm by how you write the line:

Want	pandas	Cost per lookup
rows where `equipment_id == 'AHU_1'` and `ts` in a range	boolean mask: `df[(df.equipment_id == "AHU_1") & (df.ts >= a) & (df.ts < b)]`	**O(n)**: compares every row, every time (SQL's `SCAN`)
the same, after `set_index(["equipment_id", "ts"]).sort_index()`	`df.loc[("AHU_1", slice(a, b))]`	**O(log n)**: binary search on the sorted index (SQL's `SEARCH USING INDEX`)

Want	pandas	Cost per lookup
the same on one sorted array	`np.searchsorted(ts, [a, b])`, then slice	**O(log n)**

Sorting costs O(n log n) once. After that every lookup is O(log n). That's all a B-tree index is: a sorted copy you pay for up front. So the question is always the same one: **how many lookups will you do?**

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_history(
    conn
)  # lab_history (Day 11). It comes with an index; we drop it to see
        the "before".
conn.execute("DROP INDEX IF EXISTS lab_history_eq_ts")
conn.commit()

SID = "bf422925-1e2d-598d-9069-3b0f17d49c48"  # one series (a sensor on
        AHU_1)
T0 = int(pd.Timestamp("2026-07-01", tz="UTC").timestamp())  # 2026-07-
        01 00:00 UTC, epoch seconds
T1 = T0 + 86_400  # one day later


def plan(sql: str, params: dict | tuple = ()) -> list[str]:
    """What SQLite would do for this query (the `detail` column of
        EXPLAIN QUERY PLAN)."""
    return [row[3] for row in conn.execute("EXPLAIN QUERY PLAN " + sql,
        params)]


def show(label: str, sql: str, params: dict | tuple = ()) -> None:
    for step in plan(sql, params):
        print(f"{label:<28} {step}")
```

Querying `samples`. One series for one day versus every series for one day:

```
ONE_SERIES_DAY = "SELECT * FROM samples WHERE timeseries_id = :sid AND
         ts >= :t0 AND ts < :t1"
ALL_SERIES_DAY = "SELECT * FROM samples WHERE ts >= :t0 AND ts < :t1"
p = {"sid": SID, "t0": T0, "t1": T1}
p_ts = {"t0": T0, "t1": T1}

show("one series, one day:", ONE_SERIES_DAY, p)
show("every series, one day:", ALL_SERIES_DAY, p_ts)

n_one = len(conn.execute(ONE_SERIES_DAY, p).fetchall())
n_all = len(conn.execute(ALL_SERIES_DAY, p_ts).fetchall())
print(n_one, "rows vs", n_all, "rows")
assert (n_one, n_all) == (288, 33984)
assert "SEARCH" in plan(ONE_SERIES_DAY, p)[0] and "SCAN" in
         plan(ALL_SERIES_DAY, p_ts)[0]
```

The first plan is `SEARCH samples USING PRIMARY KEY (timeseries_id=? AND ts>? AND ts<?)`: it jumps straight to 288 rows. The second is `SCAN samples`: all 4.2 million rows, to return 33,984. Time them:

```
t_seek = lab.best_time(lambda: conn.execute(ONE_SERIES_DAY,
         p).fetchall())
t_scan = lab.best_time(lambda: conn.execute(ALL_SERIES_DAY,
         p_ts).fetchall())
print(
    f"seek {t_seek * 1000:.2f} ms   scan {t_scan * 1000:.0f} ms   (scan
         returns {n_all // n_one}x the rows)"
)
```

The seek touches 288 rows; the scan reads 4.2M. The time gap is mostly that, not cleverness.

Querying `lab_history`, without and with an index. Its index is gone, so even one AHU-day scans:

```
AHU_DAY = "SELECT * FROM lab_history WHERE equipment_id = :eq AND ts >=
          :t0 AND ts < :t1"
q = {"eq": "AHU_1", "t0": T0, "t1": T1}

show("no index:", AHU_DAY, q)
sql_before = lab.query(conn, AHU_DAY, q)
t_before = lab.best_time(lambda: conn.execute(AHU_DAY, q).fetchall())

import time

t = time.perf_counter()
conn.execute("CREATE INDEX lab_history_eq_ts ON lab_history
          (equipment_id, ts)")
conn.commit()
t_create = time.perf_counter() - t  # the price of the index, paid once

show("with index:", AHU_DAY, q)
sql_after = lab.query(conn, AHU_DAY, q)
t_after = lab.best_time(lambda: conn.execute(AHU_DAY, q).fetchall())

print(
    f"{len(sql_after)} rows | no index {t_before * 1000:.1f} ms ->
          index {t_after * 1000:.2f} ms "
    f"| CREATE INDEX took {t_create * 1000:.0f} ms"
)
lab.assert_parity(
    sql_before, sql_after, keys=["equipment_id", "ts"], label="same
          rows, with and without index"
)
assert len(sql_after) == 288
assert "SCAN" in plan(AHU_DAY, q)[0] or "SEARCH" in plan(AHU_DAY, q)[0]
assert "lab_history_eq_ts" in " ".join(plan(AHU_DAY, q))
```

The plan becomes `SEARCH lab_history USING INDEX lab_history_eq_ts`. Same 288 rows either way: an index changes how fast you get the answer, never what the answer is. (The parity check proves it. Leave the index in place: Days 13–15 use it.)

The same question in pandas, three ways. Load the table once, then filter by mask, by sorted index, and by binary search on one AHU's timestamps:

```
h = lab.history(conn)  # all 71,072 rows, sorted by (equipment_id, ts)


def by_mask():
    return h[(h.equipment_id == "AHU_1") & (h.ts >= T0) & (h.ts < T1)]
        # O(n) every call


hi = h.set_index(["equipment_id", "ts"]).sort_index()  # O(n log n),
        once


def by_index():
    return hi.loc[("AHU_1", slice(T0, T1 - 1)), :]  # label slices
        include both ends, so T1 - 1


ahu1 = h[h.equipment_id == "AHU_1"]
ahu1_ts = ahu1.ts.to_numpy()  # sorted, so binary search works


def by_search():
    lo, hi_ = np.searchsorted(ahu1_ts, [T0, T1])  # first index >= T0,
        first index >= T1
    return ahu1.iloc[lo:hi_]  # O(log n) to find the bounds, then a
        slice


for name, fn in [("mask", by_mask), ("sorted index", by_index),
        ("searchsorted", by_search)]:
    got = fn().reset_index()
    got = got[[c for c in sql_after.columns]]
    lab.assert_parity(got, sql_after, keys=["equipment_id", "ts"],
        label=f"pandas {name} == SQL")

for name, fn in [("mask", by_mask), ("sorted index", by_index),
        ("searchsorted", by_search)]:
    print(f"pandas {name:<13} {lab.best_time(fn) * 1000:7.3f} ms")
```

All three return exactly SQLite's 288 rows. The mask is the pandas version of `SCAN`; the other two are the pandas version of `SEARCH USING INDEX`. (Note that `by_search` built `ahu1` with a mask first. That one-time O(n) filter is the pandas version of paying for the index up front.)

Does the gap grow with n? That's what O(n) versus O(log n) means. Same one-day lookup, on a sorted timestamp column of growing size (synthetic data, one reading every 5 minutes):

```
rows = []
for n in lab.sizes([10_000, 100_000, 1_000_000, 4_000_000]):
    ts = lab.synthetic_history(n)["ts"].to_numpy()  # sorted
    a = ts[n // 2]
    b = a + 86_400
    t_mask = lab.best_time(lambda: ts[(ts >= a) & (ts < b)])
    t_bin = lab.best_time(lambda: ts[np.searchsorted(ts, a) :
        np.searchsorted(ts, b)])
    assert ((ts >= a) & (ts < b)).sum() == np.searchsorted(ts, b) -
        np.searchsorted(ts, a) == 288
    rows.append({"n": n, "mask_ms": t_mask * 1000, "searchsorted_ms":
        t_bin * 1000})
print(pd.DataFrame(rows).round(4).to_string(index=False))
```

Read down the `mask_ms` column: ten times the rows, about ten times the time. `searchsorted_ms` barely moves. Doubling n adds one comparison to a binary search; it doubles a scan.

When does building the index pay off? Break-even is `build cost / saving per lookup`:

```
t_build_pd = lab.best_time(lambda: h.set_index(["equipment_id",
        "ts"]).sort_index())
save_pd = lab.best_time(by_mask) - lab.best_time(by_index)
save_sql = t_before - t_after
print(
    f"pandas: sort_index {t_build_pd * 1000:.0f} ms, saves {save_pd *
        1000:.2f} ms/lookup"
    f" -> pays off after ~{t_build_pd / save_pd:.0f} lookups"
    if save_pd > 0
    else "pandas: no saving at this size"
)
print(
    f"SQLite: CREATE INDEX {t_create * 1000:.0f} ms, saves {save_sql *
        1000:.2f} ms/lookup"
    f" -> pays off after ~{t_create / save_sql:.0f} lookups"
    if save_sql > 0
    else "SQLite: no saving"
)
```

A handful of lookups: don't bother, just scan. A dashboard that asks this question thousands of times: build it. An index also isn't free to keep: every new row must be inserted into the sorted structure too, which is why a historian that writes constantly keeps only the indexes its queries need.

Try it

1. Check the plan for Day 3's "bad" filter, `date(ts, 'unixepoch') = '2026-07-01'`, with a `timeseries_id`. Which part of the key does it still use? What's the pandas analogue of that mistake?
2. If dashboards often ask "all points at one moment" (`ts = ?` across every row), what index would help, and what would it cost? Try it on `lab_history`, in SQL and in pandas.

Answers

1. Wrapping a column in a function hides it from the index. The key's first column is still usable:

```
DATE_FILTER = (
    "SELECT * FROM samples WHERE timeseries_id = :sid AND date(ts,
         'unixepoch') = '2026-07-01'"
)
show("date() filter:", DATE_FILTER, {"sid": SID})

via_date = lab.query(conn, "SELECT count(*) AS n FROM (" + DATE_FILTER
         + ")", {"sid": SID})
via_range = lab.query(conn, "SELECT count(*) AS n FROM (" +
         ONE_SERIES_DAY + ")", p)

s = lab.query(conn, "SELECT ts FROM samples WHERE timeseries_id =
         :sid", {"sid": SID})
day = pd.to_datetime(s.ts, unit="s", utc=True).dt.strftime("%Y-%m-%d")
via_pandas = pd.DataFrame(
    {"n": [int((day == "2026-07-01").sum())]}
)  # converts all 35,536 rows to datetimes

lab.assert_parity(via_pandas, via_date, label="date() filter, pandas ==
         SQL")
lab.assert_parity(via_range, via_date, label="range filter == date()
         filter")
assert via_date.n[0] == 288 and len(s) == 35536
```

`SEARCH samples USING PRIMARY KEY (timeseries_id=?)`. It finds the series fast, then evaluates `date(ts)` on all 35,536 of its rows instead of 288. Same answer as the range form (parity above), just more work. The pandas analogue is converting the whole column to datetimes and comparing strings: O(n) work to answer an O(log n) question. Compare `ts` against two numbers instead.

2. An index on `lab_history(ts)`. It costs another sorted copy of the keys (for `samples(ts)` that would be a hundred-plus MB) and slows every insert. Timescale solves the "by time" access differently: it splits the table into time **chunks** (below).

```
AT_MOMENT = "SELECT * FROM lab_history WHERE ts = :t"
tt = {"t": T0 + 12 * 3600}  # noon UTC, 2026-07-01
show("ts = ?, no ts index:", AT_MOMENT, tt)
t_noidx = lab.best_time(lambda: conn.execute(AT_MOMENT, tt).fetchall())
conn.execute("CREATE INDEX lab_history_ts ON lab_history (ts)")
show("ts = ?, with ts index:", AT_MOMENT, tt)
t_idx = lab.best_time(lambda: conn.execute(AT_MOMENT, tt).fetchall())
at_sql = lab.query(conn, AT_MOMENT, tt)

by_ts = h.set_index("ts").sort_index()  # a sorted, non-unique index:
         both AHUs share each timestamp
at_pd = by_ts.loc[[tt["t"]]].reset_index()
at_mask = h[h.ts == tt["t"]].reset_index(drop=True)  # the O(n)
         alternative

lab.assert_parity(
    at_pd[at_sql.columns], at_sql, keys=["equipment_id", "ts"],
         label="ts = ?, index == SQL"
)
lab.assert_parity(
    at_mask[at_sql.columns], at_sql, keys=["equipment_id", "ts"],
         label="ts = ?, mask == SQL"
)
print(f"SQLite {t_noidx * 1000:.1f} ms -> {t_idx * 1000:.3f} ms")
assert len(at_sql) == 2  # one row per AHU
conn.execute("DROP INDEX lab_history_ts")  # keep the lab tidy
conn.commit()
```

Why it matters

The same thinking applies to triple stores (week 5). A graph of `(subject, predicate, object)` triples needs several sorted indexes (SPO, POS, OSP) because SPARQL queries fix different positions. Which index a query can use is the whole performance story. And it's the same story in pandas: `searchsorted` and `.loc` on a sorted index are what an index is, and a boolean mask over a big frame is a scan you chose to run.

Day 13 — Window functions: LAG, rolling averages, streaks

Goal: compare each row with its neighbours in time, without collapsing rows like `GROUP BY` does. SQL window functions and pandas `shift` / `rolling` / `cumsum` are the same ideas; do each both ways and check they agree.

Read

A **window function** computes a value for each row from a "window" of related rows. All rows stay in the result.

```
LAG(x)        OVER (PARTITION BY equipment_id ORDER BY ts)   -- x from
the previous row
LEAD(x)       OVER (...)                                      -- x from
the next row
ROW_NUMBER()  OVER (PARTITION BY ... ORDER BY ts)             -- 1, 2,
3 ... within each partition
AVG(x)        OVER (PARTITION BY ... ORDER BY ts
                    ROWS BETWEEN 11 PRECEDING AND CURRENT ROW) --
rolling mean of the last 12 rows (1 hour)
SUM(x)        OVER (PARTITION BY ... ORDER BY ts)             --
running total
```

- `PARTITION BY equipment_id` keeps AHU_1 and AHU_2 separate. Without it, AHU_2's first row would "follow" AHU_1's last.
- `ORDER BY ts` defines what "previous" means.

pandas has a one-to-one map. `PARTITION BY` becomes `groupby`, and `ORDER BY` becomes "the frame is already sorted" (`lab.history()` returns rows sorted by `(equipment_id, ts)`):

SQL	pandas
`LAG(x) OVER (PARTITION BY e ORDER BY ts)`	`df.groupby("e")["x"].shift()`
`LEAD(x) OVER (...)`	`df.groupby("e")["x"].shift(-1)`
`AVG(x) OVER (... ROWS BETWEEN 11 PRECEDING AND CURRENT ROW)`	`df.groupby("e")["x"].rolling(12, min_periods=1).mean()`
`SUM(x) OVER (PARTITION BY e ORDER BY ts)`	`df.groupby("e")["x"].cumsum()`
`ROW_NUMBER() OVER (PARTITION BY e, s ORDER BY ts)`	`df.groupby(["e", "s"]).cumcount() + 1`
first row's `LAG` is NULL	first row's `shift()` is NaN

Two details that bite: `rolling(12)` alone leaves the first 11 rows NaN (it wants a full window), while SQL averages whatever rows exist, so ask for `min_periods=1` to match. And `shift()` turns an integer column into floats (NaN needs a float), harmless here.

Streaks (gaps and islands). Fault rules need "this condition has been true for N rows in a row". The standard trick takes three steps, in either language:

1. Mark where the value changes: `CASE WHEN x = LAG(x) OVER w THEN 0 ELSE 1 END AS new_streak`, or `x.ne(x.groupby(eq).shift())` in pandas.
2. A running sum of those marks numbers each streak: `SUM(new_streak) OVER w AS streak_id`, or `.cumsum()`.
3. Number the rows inside each streak: `ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id ...)`, or `cumcount()`.

A window function can't be used directly in `WHERE`, because windows are computed after filtering. Wrap the query in a subquery, or a `WITH name AS (...)` common table expression (CTE), and filter outside. (pandas has no such rule: a column is just a column, so you filter it right away.)

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_history(
    conn
)  # lab_history: wide SI table with an (equipment_id, ts) index (Day
        11, Day 12)
h = lab.history(conn)  # the same table as a DataFrame, sorted by
        (equipment_id, ts)
eq = h["equipment_id"]
```

Units are SI now (Day 11), so duct static pressure is in pascals. **LAG: each reading next to the previous one.**

```
LAG_SQL = """
SELECT equipment_id, ts, duct_static,
       LAG(duct_static) OVER (PARTITION BY equipment_id ORDER BY ts) AS
         prev
FROM lab_history
"""
lag_sql = lab.query(conn, LAG_SQL)

prev_static = h.groupby("equipment_id")["duct_static"].shift()
lag_pd = h[["equipment_id", "ts",
         "duct_static"]].assign(prev=prev_static)
lab.assert_parity(lag_pd, lag_sql, keys=["equipment_id", "ts"],
         label="LAG(duct_static)")

first = lag_pd[lag_pd.equipment_id == "AHU_1"].head(3)
print(first.assign(t=pd.to_datetime(first.ts, unit="s",
         utc=True)).to_string(index=False))
assert (
    lag_pd[lag_pd.equipment_id == "AHU_1"].prev.isna().sum() == 1
)  # only the first row has no previous
```

The first row's `prev` is NULL in SQL and NaN in pandas: nothing came before it. The parity check treats the two as equal, and the second AHU's first row is NULL too, because the window is partitioned.

Fan starts: off in the previous row, on in this one.

```
STARTS_SQL = """
WITH w AS (
  SELECT equipment_id, ts, fan_status,
         LAG(fan_status) OVER (PARTITION BY equipment_id ORDER BY ts)
         AS prev
  FROM lab_history
)
SELECT equipment_id, count(CASE WHEN fan_status = 1 AND prev = 0 THEN 1
         END) AS starts
FROM w
GROUP BY equipment_id
ORDER BY equipment_id
"""
starts_sql = lab.query(conn, STARTS_SQL)

prev_fan = h.groupby("equipment_id")["fan_status"].shift()
starts_pd = ((h.fan_status == 1) & (prev_fan ==
         0)).groupby(eq).sum().rename("starts").reset_index()
lab.assert_parity(starts_pd, starts_sql, label="fan starts")
print(starts_sql.to_string(index=False))
assert starts_sql.set_index("equipment_id").loc["AHU_1", "starts"] ==
         123
```

About one start a day for 123 days, as you'd expect for a scheduled AHU. (A NaN or NULL `prev` makes the comparison false in both languages, so the very first row can never count as a start.)

Longest continuous run of each fan, in hours: the streak trick.

```
RUN_SQL = """
WITH marks AS (
  SELECT equipment_id, ts, fan_status,
         CASE WHEN fan_status = LAG(fan_status) OVER (PARTITION BY
         equipment_id ORDER BY ts)
              THEN 0 ELSE 1 END AS new_streak
  FROM lab_history
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
)
SELECT equipment_id, max(n) * 5 / 60.0 AS longest_run_h
FROM (SELECT equipment_id, streak_id, count(*) AS n FROM streaks WHERE
         fan_status = 1
      GROUP BY equipment_id, streak_id)
GROUP BY equipment_id
ORDER BY equipment_id
"""
run_sql = lab.query(conn, RUN_SQL)

new_streak = h.fan_status.ne(h.fan_status.groupby(eq).shift()).astype(
    int
)  # step 1: where it changes
streak_id = new_streak.groupby(eq).cumsum()  # step 2: number the
         streaks
on = h[h.fan_status == 1].assign(streak_id=streak_id)  # step 3 is just
         a row count per streak
rows_per_run = on.groupby(["equipment_id", "streak_id"]).size()
run_pd = (
    rows_per_run.groupby("equipment_id").max().mul(5 /
         60).rename("longest_run_h").reset_index()
)
lab.assert_parity(run_pd, run_sql, label="longest fan run")
print(run_sql.round(1).to_string(index=False))
assert round(run_sql.set_index("equipment_id").loc["AHU_1",
         "longest_run_h"], 1) == 109.4
```

109 hours: the fan ran for more than four days straight. For an office AHU that's worth asking about (an override? a schedule left in occupied?). It's the same question Day 9's unoccupied runtime raised.

Rolling average: a 1-hour (12-row) mean of duct static, for every row.

```
ROLL_SQL = """
SELECT equipment_id, ts,
       AVG(duct_static) OVER (PARTITION BY equipment_id ORDER BY ts
                              ROWS BETWEEN 11 PRECEDING AND CURRENT
         ROW) AS static_1h
FROM lab_history
"""
roll_sql = lab.query(conn, ROLL_SQL)

roll = h.groupby("equipment_id")["duct_static"].rolling(12,
         min_periods=1).mean()
roll_pd = h[["equipment_id",
         "ts"]].assign(static_1h=roll.reset_index(level=0, drop=True))
lab.assert_parity(roll_pd, roll_sql, keys=["equipment_id", "ts"],
         label="1 h rolling mean")
```

SQLite keeps a running sum it adds to and subtracts from, while pandas uses its own online update, so the two differ in the last few bits. That's why `assert_parity` compares with a tolerance, never with `==`.

Why vectorized? SQL and pandas both walk the sorted rows once, O(n). The difference between them and a Python `for` loop is the constant: the loop runs the interpreter 71,072 times, the others run compiled code. This loop does almost nothing per row, so it is only a few times slower; put real logic in the loop body and the gap grows to 100x.

```
def lag_loop(equips, values):
    out = [float("nan")] * len(values)
    for i in range(1, len(values)):
        if equips[i] == equips[i - 1]:  # same partition?
            out[i] = values[i - 1]
    return out


loop_prev = lag_loop(eq.tolist(), h.duct_static.tolist())
lab.assert_parity(
    pd.DataFrame({"prev": loop_prev}), pd.DataFrame({"prev":
         prev_static}), label="loop == shift"
)

t_sql = lab.best_time(lambda: lab.query(conn, LAG_SQL))
t_shift = lab.best_time(lambda: h.groupby("equipment_id")
         ["duct_static"].shift())
t_loop = lab.best_time(lambda: lag_loop(eq.tolist(),
         h.duct_static.tolist()))
print(
    f"LAG over {len(h):,} rows: SQL {t_sql * 1000:.0f} ms (includes
         fetching) | "
    f"pandas shift {t_shift * 1000:.1f} ms | python loop {t_loop *
         1000:.0f} ms"
)
```

pandas looks far faster than SQL here only because the data is already in memory; the SQL time includes reading the table and building a DataFrame out of it. Compare like with like: both are the same order of work.

Try it

1. A 1-hour rolling average of AHU_2's duct static pressure. What's its maximum, in pascals?
2. Count AHU_1 fan **stops** (on, then off). Should it equal the starts? Why might it differ by one?

Answers

1. Filter the rows to AHU_2 first (the inner query's `WHERE` in SQL, a mask in pandas), then window.

```
MAX_ROLL_SQL = """
SELECT max(r) AS max_static_1h_pa FROM (
  SELECT AVG(duct_static) OVER (ORDER BY ts ROWS BETWEEN 11 PRECEDING
         AND CURRENT ROW) AS r
  FROM lab_history WHERE equipment_id = 'AHU_2'
)
"""
max_sql = lab.query(conn, MAX_ROLL_SQL)
ahu2 = h[h.equipment_id == "AHU_2"]
max_pd = pd.DataFrame(
    {"max_static_1h_pa": [ahu2.duct_static.rolling(12,
         min_periods=1).mean().max()]}
)
lab.assert_parity(max_pd, max_sql, label="max rolling static, AHU_2")
print(round(max_sql.iloc[0, 0], 1), "Pa")
assert round(max_sql.iloc[0, 0], 1) == 417.3
```

About 417 Pa (1.675 inH2O, the number this lesson used to print before the history became SI).

2. Same shape as the starts, with the comparison flipped:

```
STOPS_SQL = """
WITH w AS (
  SELECT fan_status, LAG(fan_status) OVER (ORDER BY ts) AS prev
  FROM lab_history WHERE equipment_id = 'AHU_1'
)
SELECT count(*) AS stops FROM w WHERE fan_status = 0 AND prev = 1
"""
stops_sql = lab.query(conn, STOPS_SQL)
f1 = h.loc[h.equipment_id == "AHU_1", "fan_status"]
stops_pd = pd.DataFrame({"stops": [int(((f1 == 0) & (f1.shift() ==
         1)).sum())]})
lab.assert_parity(stops_pd, stops_sql, label="AHU_1 fan stops")
assert stops_sql.stops[0] == 122
```

122 stops against 123 starts. They can differ by one because the data can begin or end in the middle of a run (it ends with the fan running).

Why it matters

“Condition true for N consecutive rows” is how a fault rule ignores a one-sample blip and reports only persistent problems. Tomorrow that’s the confirmation step of a real rule, written once in SQL and once in pandas, with a parity check between them.

Day 14 — Write a fault rule (FC1, duct static too low) twice, and check it against Open-FDD

Goal: turn an engineering rule into SQL *and* into pandas, prove the two agree row for row, count fault hours, and compare with a production FDD engine's results.

Read

Fault detection and diagnostics (FDD) runs rules over trend data to find equipment that isn't doing what it should. Open-FDD's rules come from ASHRAE Guideline 36 and the NIST/PNNL APAR rules. **FC1** says:

> The supply fan is running (at or near full speed) but duct static pressure stays below its setpoint.

The fan is working as hard as it can and still can't make pressure. Likely causes: a broken belt, a failing VFD, a blocked filter or coil, a leaky duct, or a setpoint too high for this system.

Every Open-FDD rule (`sql/fdd/postgres/rules/*.sql`, and its pandas twin in `open_fdd/rules`) has the same shape:

1. **Gate**: only judge rows where the equipment is on (`fan_status = 1`). Off-hours aren't faults.
2. **Raw fault**: `CASE WHEN <condition> THEN 1 ELSE 0 END`, with a tolerance so noise doesn't trip it.
3. **Confirm**: count a row only once the raw fault has held for N consecutive rows (Day 13's streaks).
4. **Aggregate**: fault rows × 5 minutes = fault hours.

The math is **SI**, like Open-FDD: `lab_history` (Day 11) already holds pressure in pascals. The tolerance is **25 Pa** (about 0.1 inH2O) and a fault is confirmed after **2 rows** (10 minutes).

Open-FDD keeps every rule twice on purpose. SQL is what runs next to the data. pandas is the **oracle**: a second, independent implementation, easy to step through in a notebook. Its parity tests (`tests/parity` in Open-FDD) run both on the same synthetic data and fail if a single row differs. You'll do exactly that here.

How each step maps across:

Step	SQL (per row, inside window functions)	pandas (per column, vectorized)
gate + raw fault	`CASE WHEN fan_status = 1 AND ... THEN 1 ELSE 0 END`	boolean mask (`h.fan_status == 1) & (...)`)
“value changed”	`raw = LAG(raw) OVER (PARTITION BY equipment_id ORDER BY ts)`	`raw.ne(raw.groupby(eq).shift())`
streak id	`SUM(new_streak) OVER (... ORDER BY ts)`	`new.groupby(eq).cumsum()`
row within streak	`ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id ...)`	`groupby([eq, streak]).cumcount() + 1`
hours	`sum(fault) * 5 / 60.0`	`fault.sum() * 5 / 60`

Both are O(n) once the rows are in `(equipment_id, ts)` order. SQLite gets that order from the `lab_history (equipment_id, ts)` index; pandas from the `ORDER BY` in `lab.history()`.

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_history(conn)  # lab_history: one row per AHU per 5 minutes,
         SI units (Day 11)

TOL_PA = 25.0  # tolerance below setpoint, pascals (~0.1 inH2O)
CONFIRM_ROWS = 2  # 2 rows x 5 min = 10 minutes before a fault counts
```

The rule in SQL. One string, four CTEs, one per step above. Named parameters (`:tol`, `:confirm`) keep numbers out of the SQL text, as Open-FDD’s `render_rule.py` does with its `{{...}}` placeholders.

```
FC1_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END AS raw_fault
  FROM lab_history
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts)
                THEN 0 ELSE 1 END AS new_streak
  FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len
  FROM streaks
)
SELECT equipment_id, ts, raw_fault, streak_id,
       CASE WHEN raw_fault = 1 AND streak_len >= :confirm THEN 1 ELSE 0
         END AS fault
FROM ranked
ORDER BY equipment_id, ts
"""
params = {"tol": TOL_PA, "confirm": CONFIRM_ROWS}
fc1_sql = lab.query(conn, FC1_SQL, params)
print(fc1_sql.head())
```

The same rule in pandas. Each line is one column operation over all 71,072 rows; `groupby(eq)` plays the part of `PARTITION BY equipment_id`.

```
h = lab.history(conn)  # sorted by equipment_id, ts
eq = h["equipment_id"]

raw = ((h["fan_status"] == 1) & (h["duct_static"] < h["duct_static_sp"]
         - TOL_PA)).astype(int)
new_streak = raw.ne(raw.groupby(eq).shift()).astype(int)  # first row:
         shift() is NaN, so "new"
streak_id = new_streak.groupby(eq).cumsum()
streak_len = raw.groupby([eq, streak_id]).cumcount() + 1
fault = ((raw == 1) & (streak_len >= CONFIRM_ROWS)).astype(int)

fc1_pd = h[["equipment_id", "ts"]].assign(raw_fault=raw,
         streak_id=streak_id, fault=fault)
lab.assert_parity(fc1_pd, fc1_sql, keys=["equipment_id", "ts"],
         label="FC1 per row")
```

Every one of the 71,072 rows agrees: same raw flag, same streak number, same confirmed fault. Now the summary, again both ways:

```
lab.to_sqlite(fc1_sql, conn, "lab_fc1", index=["equipment_id", "ts"])
        # Day 15 reads it
SUMMARY_SQL = """
SELECT equipment_id,
       sum(raw_fault) * 5 / 60.0 AS raw_fault_h,
       sum(fault) * 5 / 60.0     AS confirmed_fault_h,
       count(DISTINCT CASE WHEN fault = 1 THEN streak_id END) AS
         episodes
FROM lab_fc1
GROUP BY equipment_id
ORDER BY equipment_id
"""
summary_sql = lab.query(conn, SUMMARY_SQL)

hours = fc1_pd.groupby("equipment_id")[["raw_fault",
        "fault"]].sum().mul(5 / 60)
episodes = fc1_pd[fc1_pd.fault == 1].groupby("equipment_id")
        ["streak_id"].nunique()
summary_pd = (
    hours.rename(columns={"raw_fault": "raw_fault_h", "fault":
        "confirmed_fault_h"})
    .assign(episodes=episodes)
    .fillna({"episodes": 0})
    .reset_index()
)
lab.assert_parity(summary_pd, summary_sql, label="FC1 summary")
print(summary_sql.round(2).to_string(index=False))
```

Notice the parity check compares the **unrounded** hours and the printout rounds. That's deliberate. AHU_1 has 1,371 confirmed rows = 114.25 h exactly, a tie at one decimal. SQLite's `round(114.25, 1)` rounds halves away from zero (114.3); numpy and Python round halves to even (114.2). Round only for display, never before comparing.

```
ahu1 = summary_sql.set_index("equipment_id").loc["AHU_1"]
print(ahu1.to_dict())
assert (ahu1.raw_fault_h, ahu1.confirmed_fault_h, ahu1.episodes) ==
        (120.0, 114.25, 53)
```

AHU_1 has about 114 confirmed fault hours in 53 episodes. AHU_2 has 1.5 hours: essentially healthy for this rule (its problem is the opposite: static *above* setpoint, Day 8).

Compare with Open-FDD. Its results for this data ship in `data/BUILDING_50/*/fdd_events.csv`. `lab.ensure_events` loads them into `lab_fdd_event` (one row per fault episode: `equipment_id`, `rule_name`, `start_ts`, `end_ts`, `duration_min`).

```
lab.ensure_events(conn)
ofdd = lab.query(
    conn,
    """SELECT rule_name, equipment_id, count(*) AS episodes,
         round(sum(duration_min) / 60, 1) AS hours
       FROM lab_fdd_event WHERE rule_name = 'fc1' GROUP BY rule_name,
         equipment_id""",
)
print(ofdd.to_string(index=False))
assert ofdd.iloc[0].tolist() == ["fc1", "AHU_1", 80, 484.1]
```

80 episodes and 484 hours, against our 53 and 114. Different numbers don't mean either is wrong yet. Check whether the two engines are finding **the same events**: how many of Open-FDD's episodes contain at least one of our confirmed fault rows?

In SQL that's a **correlated `EXISTS`**: for each of the E events, probe our fault rows for that AHU and time range. With the `(equipment_id, ts)` index each probe is a B-tree range seek, so the whole thing is O(E log N).

```
OVERLAP_SQL = """
SELECT count(*) AS openfdd_episodes,
       sum(EXISTS (SELECT 1 FROM lab_fc1 f
                   WHERE f.equipment_id = e.equipment_id AND f.fault =
         1
                     AND f.ts BETWEEN e.start_ts AND e.end_ts)) AS
         also_found_by_us
FROM lab_fdd_event e
WHERE e.rule_name = 'fc1'
"""
overlap_sql = lab.query(conn, OVERLAP_SQL)
```

pandas has no `EXISTS`. The fast equivalent is **binary search**: sort our fault timestamps once, then `np.searchsorted` finds, for every event at once, how many fault rows fall before `start_ts` and how many at or before `end_ts`. If those differ, the event contains one. Same O(E log F) as the index seek, written as array code.

```
events = lab.query(
    conn, "SELECT equipment_id, start_ts, end_ts FROM lab_fdd_event
        WHERE rule_name = 'fc1'"
)
found = np.zeros(len(events), dtype=bool)
for equip, ev in events.groupby("equipment_id"):
    t = np.sort(fc1_pd.loc[(fc1_pd.equipment_id == equip) &
        (fc1_pd.fault == 1), "ts"].to_numpy())
    lo = np.searchsorted(t, ev["start_ts"].to_numpy(), side="left")
    hi = np.searchsorted(t, ev["end_ts"].to_numpy(), side="right")
    found[ev.index] = hi > lo
overlap_pd = pd.DataFrame(
    {"openfdd_episodes": [len(events)], "also_found_by_us":
        [int(found.sum())]}
)

lab.assert_parity(overlap_pd, overlap_sql, label="overlap with Open-
        FDD")
print(overlap_sql.to_string(index=False))
assert overlap_sql.iloc[0].tolist() == [80, 73]
```

73 of 80 overlap. Most of Open-FDD's episodes overlap ours: same problems, measured differently. Look at the episode lengths (`max(duration_min)` is about 1,425 minutes, nearly a day). Open-FDD's episode clock runs from first to last fault sample, including stretches in between, and its event timestamps aren't on this 5-minute grid. It ran on its own copy of the data with its own tuning. Totals depend on those choices. Overlap tells you whether the rules agree on **what** is wrong.

Real Open-FDD rules (`cmd1_fan_mismatch.sql` is a short one to read) add one more step you don't need here: **forward sample-and-hold with a gap limit** (`MAX_GAP_HOURS`). Each row's state is held until the next sample, but a gap longer than the limit counts as *unknown*: not fault time, and it breaks the streak. Building 50 has no gaps (every step is exactly 300 s), so counting rows works. Day 18 builds the gap-aware version.

Try it

1. Change the tolerance from 25 Pa to 50 Pa (~0.2 inH2O). How much do AHU_1's confirmed hours drop? Do it in SQL by changing only the parameter, and in pandas by changing only the tolerance, and check parity again.
2. Write the opposite rule, **duct static too high** (`duct_static > duct_static_sp + 25 Pa` while running), and run it on AHU_2 both ways.

Answers

A small function makes the pandas rule reusable for any raw condition; the SQL just gets a new parameter or a flipped comparison.

```
def confirmed(h, raw, n=CONFIRM_ROWS):
    """Per-row confirmed fault (0/1) from a raw 0/1 Series aligned with
        h (sorted by equipment, ts)."""
    eq = h["equipment_id"]
    streak =
        raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    return ((raw == 1) & (raw.groupby([eq, streak]).cumcount() + 1 >=
        n)).astype(int)


# 1. tolerance 50 Pa
sql_50 = lab.query(conn, FC1_SQL, {"tol": 50.0, "confirm":
        CONFIRM_ROWS})
raw_50 = ((h.fan_status == 1) & (h.duct_static < h.duct_static_sp -
        50.0)).astype(int)
pd_50 = h[["equipment_id", "ts"]].assign(fault=confirmed(h, raw_50))
lab.assert_parity(pd_50, sql_50[["equipment_id", "ts", "fault"]],
        label="FC1 at 50 Pa")
hours_50 = round(sql_50.loc[sql_50.equipment_id == "AHU_1",
        "fault"].sum() * 5 / 60, 1)
print("AHU_1 confirmed FC1 hours at 50 Pa:", hours_50)

# 2. duct static too high, AHU_2
HIGH_SQL = FC1_SQL.replace(
    "duct_static < duct_static_sp - :tol", "duct_static >
        duct_static_sp + :tol"
)
high_sql = lab.query(conn, HIGH_SQL, params)
raw_hi = ((h.fan_status == 1) & (h.duct_static > h.duct_static_sp +
        TOL_PA)).astype(int)
high_pd = h[["equipment_id", "ts"]].assign(fault=confirmed(h, raw_hi))
lab.assert_parity(high_pd, high_sql[["equipment_id", "ts", "fault"]],
        label="static too high")
hi_h = round(high_sql.loc[high_sql.equipment_id == "AHU_2",
        "fault"].sum() * 5 / 60, 1)
print("AHU_2 confirmed high-static hours:", hi_h)
assert (hours_50, hi_h) == (101.2, 1428.3)
```

1. From about 114 to about 101 hours. Most of AHU_1's shortfall is more than 50 Pa: a real problem, not a marginal one.
2. About 1,428 hours, nearly all of AHU_2's 1,500 run hours. The fan pushes far more pressure than its setpoint asks for, all the time: wasted fan energy, and possibly a reset sequence that isn't working. Open-FDD's version of this rule is `ahu_ducthi.sql`.

Why it matters

Notice what the rule needed from outside the data: which column is "duct static" and which is "fan status" for each AHU. Change buildings and only that mapping changes. That's the case for a semantic model. The rule stays the same, and the model tells it where the inputs are. And because you have the rule twice, a change to either one that alters a single row fails the parity check before it ships.

Day 15 — What production does differently (Timescale), and the UUID bridge to RDF

Goal: know which parts of Unit 1 carry over to a production historian, see where the RDF model picks up, and run one fault rule end to end with the points found by the **model** rather than by a table you wrote.

Read

SQLite here, Timescale in production

Open-FDD's production stack keeps history in **PostgreSQL + TimescaleDB**. Everything you wrote this unit is plain SQL and carries over. The differences are operational:

Need	This course (SQLite)	Timescale
Store readings	`samples(timeseries_id, ts, value)`, key on series + time	a **hypertable**: same columns, split into time **chunks** automatically, so "last week" only touches a few chunks
Bucket by time	`strftime('%Y-%m', ts, 'unixepoch')`, `ts / 3600 * 3600`	`time_bucket('1 hour', time)`
Summary tables	`CREATE TABLE lab_hourly AS ...`, rebuilt by hand	**continuous aggregates**, refreshed incrementally as data arrives
Old data	keep everything	**compression** and **retention policies** per chunk
Many writers and readers	one writer at a time	a server with concurrent connections and roles
Wide `history` input	`lab_history` table	a view or temporary table built for one site, then the rule SQL runs as is

The pandas side of "bucket by time" is the same idea again: `dt.strftime("%Y-%m")`, `dt.to_period("M")`, or `resample`. The rules don't change. The Open-FDD SQL you read yesterday runs on Postgres with small dialect changes (`sql/fdd/README.md` lists them). SQLite needs a few more, like sample standard deviation, which it lacks.

The UUID bridge

All unit long you used `lab_point` to know that `52422584-…` is AHU_1's duct static pressure. The API doesn't use a table for that. It uses the RDF model, `model/building_50.ttl`. Open it in a text editor and search for `52422584-9894-50bd-9696-2fbb8b062518`:

```
bldg:AHU_1_DA_P a bts:Entity ;
    rdfs:label "AHU1 DA-P" ;
    ref:hasExternalReference [ a ref:TimeseriesReference ;
            ref:hasTimeseriesId "52422584-9894-50bd-9696-2fbb8b062518"
;
            ref:storedAt bldg:timeseries_db ] ;
    bts:hasHaystackMarker "air", "discharge", "his", ...
```

Read it as statements, each one **subject, predicate, object**:

- `AHU_1_DA_P` has label "AHU1 DA-P".
- `AHU_1_DA_P` has an external reference, a TimeseriesReference with id `52422584-…`, stored at `bldg:timeseries_db`.
- `AHU_1_DA_P` has the Haystack markers `air`, `discharge`, …

And further down, `bldg:timeseries_db` describes the database you've been querying, including its schema: `samples(timeseries_id TEXT, ts INTEGER epoch-s UTC, value REAL)`.

So the model is a graph of statements about things, and one kind of statement points into the time-series store by UUID. Units 2–4 teach you to store, query and build graphs like this. Then the question "give me duct static and fan status for every AHU" is answered by the model. The answer comes back as UUIDs, and those go into the SQL you already know. The last section of this lesson does exactly that, through the repo's real API.

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_history(conn)  # lab_history: one row per AHU per 5 minutes,
        SI units (Day 11)
h = lab.history(conn)  # sorted by equipment_id, ts
```

FC1 again, compactly. This lesson runs on its own, so rebuild Day 14's rule. The SQL is one template where only the raw condition changes (`__RAW__`); the pandas side is one function. Both are the four steps from Day 14: gate and raw fault, streak id, row within streak, confirm.

```
RULE_SQL = """
WITH base AS (
  SELECT equipment_id, ts, CASE WHEN __RAW__ THEN 1 ELSE 0 END AS
         raw_fault FROM lab_history
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts)
                THEN 0 ELSE 1 END AS new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len FROM streaks
)
SELECT equipment_id, ts, raw_fault, streak_id,
       CASE WHEN raw_fault = 1 AND streak_len >= :confirm THEN 1 ELSE 0
         END AS fault
FROM ranked ORDER BY equipment_id, ts
"""
FC1_SQL = RULE_SQL.replace("__RAW__", "fan_status = 1 AND duct_static <
         duct_static_sp - :tol")


def rule_pandas(h: pd.DataFrame, raw: pd.Series, confirm: int) ->
         pd.DataFrame:
    """Day 14's logic: raw 0/1 -> streak id -> row-in-streak ->
         confirmed fault. h is sorted by (equipment, ts)."""
    eq = h["equipment_id"]
    raw = raw.astype(int)
    streak_id =
         raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    streak_len = raw.groupby([eq, streak_id]).cumcount() + 1
    fault = ((raw == 1) & (streak_len >= confirm)).astype(int)
    return h[["equipment_id", "ts"]].assign(raw_fault=raw,
         streak_id=streak_id, fault=fault)


TOL_PA, CONFIRM_ROWS = 25.0, 2
fc1_sql = lab.query(conn, FC1_SQL, {"tol": TOL_PA, "confirm":
         CONFIRM_ROWS})
fc1_pd = rule_pandas(
    h, (h.fan_status == 1) & (h.duct_static < h.duct_static_sp -
         TOL_PA), CONFIRM_ROWS
)
lab.assert_parity(fc1_pd, fc1_sql, keys=["equipment_id", "ts"],
         label="FC1 per row")

lab.to_sqlite(fc1_sql, conn, "lab_fc1", index=["equipment_id", "ts"])
hours = fc1_sql.groupby("equipment_id").fault.sum() * 5 / 60
print(hours.to_dict())
assert hours["AHU_1"] == 114.25  # Day 14's number
```

What the course added to the database. The catalogue is itself a table, `sqlite_master`:

```
lab_objects = lab.query(
    conn, "SELECT type, name FROM sqlite_master WHERE name LIKE
        'lab\\_%' ESCAPE '\\' ORDER BY 1, 2"
)
print(lab_objects.to_string(index=False))
assert {"lab_point", "lab_history", "lab_fc1"} <= set(lab_objects.name)
```

The one-line summary of Unit 1: confirmed FC1 hours per AHU per month, in SQL with `strftime`, and in pandas by turning epoch seconds into datetimes.

```
MONTHLY_SQL = """
SELECT equipment_id, strftime('%Y-%m', ts, 'unixepoch') AS month,
       sum(fault) * 5 / 60.0 AS fc1_hours
FROM lab_fc1
GROUP BY equipment_id, month
ORDER BY equipment_id, month
"""
monthly_sql = lab.query(conn, MONTHLY_SQL)

when = pd.to_datetime(fc1_pd.ts, unit="s", utc=True)  # epoch seconds -
        > UTC datetimes
monthly_pd = (
    fc1_pd.assign(month=when.dt.strftime("%Y-%m"))  # the same format
        string as SQLite's strftime
    .groupby(["equipment_id", "month"])["fault"]
    .sum()
    .mul(5 / 60)
    .rename("fc1_hours")
    .reset_index()
)
lab.assert_parity(monthly_pd, monthly_sql, label="FC1 hours per month")

# the other idiom: a Period is "the month this timestamp falls in"
        (drop the timezone first)
period = when.dt.tz_localize(None).dt.to_period("M").astype(str)
assert (period == when.dt.strftime("%Y-%m")).all()

print(monthly_sql.pivot(index="month", columns="equipment_id",
        values="fc1_hours").round(1))
assert abs(monthly_sql[monthly_sql.equipment_id ==
        "AHU_1"].fc1_hours.sum() - 114.25) < 1e-9
```

Notice `GROUP BY equipment_id, month` uses the alias from the `SELECT`: SQLite allows that, standard SQL doesn't. In pandas you simply group on the column you just made. Both are O(n): one pass over the fault rows.

When you're done, `uv run python lessons/course.py reset` drops every `lab_*` object. The API's data is untouched.

The model-driven run: FC1 through the real API

Everything above found "duct static" and "fan status" with `lab_point`, a table built from CSVs. Production doesn't. Here is the same rule with every point found by **SPARQL against the RDF model**, and every series fetched through the app's **HTTP API**, with no ids written in the code. `lab.Api` runs the repo's FastAPI app in-process (no server, no network) on the same database. Pattern order matters to rdflib's simple engine: starting from the 118 points that have a timeseries reference keeps every later join small (Day 6 explains why).

```
FC1_POINTS_SPARQL = """
PREFIX bts: <https://example.org/openfdd/bldgts#>
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?ahu ?role ?point ?timeseriesId WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseriesId .
  ?point bts:hasHaystackTag ?roleTag .
  ?roleTag bts:tagName "ofddRole" ; bts:tagValue ?role .
  FILTER(?role IN ("fan-status", "duct-static-pressure", "duct-static-
        pressure-sp"))
  ?point bts:hasHaystackTag ?direct .
  ?direct bts:tagName "equipRef" ; bts:tagValue ?ahu .
}
ORDER BY ?ahu ?role
"""
with lab.Api() as api:
    found = api.sparql(FC1_POINTS_SPARQL)  # one row per (AHU, role):
         the point and its UUID
    found["equipment_id"] = found.ahu.str.rsplit("#", n=1).str[-1]  #
         IRI local name: AHU_1
    found["point_id"] = found.point.str.rsplit("#", n=1).str[-1]
    series = {
        (r.equipment_id, r.role): api.timeseries(r.point_id)  # GET
         /api/points/{id}/timeseries
        for r in found.itertuples()
    }
print(found[["equipment_id", "role", "point_id",
         "timeseriesId"]].to_string(index=False))
assert len(found) == 6  # 2 AHUs x 3 roles, found by the model
assert all(
    s.attrs["timeseries_id"] == r.timeseriesId for s, r in
         zip(series.values(), found.itertuples())
)
```

The model said which UUID is which role on which AHU, and the API returned each series from that point's name; the `timeseries_id` the API reports back is the UUID the model gave us. Now the same steps as Day 11, in pandas: long, pivot to wide, convert to SI (the history from the API is as logged: inH2O).

```
long = pd.concat(
    [s.assign(equipment_id=eq_id, role=role) for (eq_id, role), s in
         series.items()],
    ignore_index=True,
)
wide = long.pivot(index=["equipment_id", "ts"], columns="role",
         values="value").reset_index()
api_h = (
    pd.DataFrame(
        {
            "equipment_id": wide.equipment_id,
            "ts": wide.ts,
            "fan_status": wide["fan-status"],
            "duct_static": lab.inh2o_to_pa(wide["duct-static-
         pressure"]),
            "duct_static_sp": lab.inh2o_to_pa(wide["duct-static-
         pressure-sp"]),
        }
    )
    .sort_values(["equipment_id", "ts"])
    .reset_index(drop=True)
)
print(len(api_h), "rows from the API")

raw_api = (api_h.fan_status == 1) & (api_h.duct_static <
         api_h.duct_static_sp - TOL_PA)
fc1_api = rule_pandas(api_h, raw_api, CONFIRM_ROWS)

lab.assert_parity(
    fc1_api, fc1_sql, keys=["equipment_id", "ts"], label="API -> pandas
         FC1 == SQL FC1"
)
lab.assert_parity(
    fc1_api, fc1_pd, keys=["equipment_id", "ts"], label="API -> pandas
         FC1 == sqlite -> pandas FC1"
)
```

Three routes to the same answer: SQL on the `lab_history` table, pandas on a DataFrame read from SQLite, and pandas on series fetched over HTTP after the model located them. Every row agrees. That is what "model-driven" means: change building and the SPARQL finds that building's points, and the rule doesn't change.

Unit 1 challenge

Write a **discharge air temperature** rule for AHU_1 and AHU_2: while the fan runs, discharge air temperature (`sat`) is more than **5 °F** away from its setpoint (`sat_sp`) for at least 30 minutes. The history is in SI, so convert the threshold. Mind the trap from Day 11: 5 °F is a temperature **difference**, so it is 5 × 5/9 = 2.78 °C, **not** `(5 - 32) * 5 / 9`. Then:

1. Exclude dropout rows (`sat` logged as 0 °F) and explain how many rows that removed.

2. Report fault hours per AHU per month, in SQL and in pandas, and check parity.
3. Pick the worst day and chart `sat`, `sat_sp` and `fan_status` for it. DB Browser's **Plot** tab works, or matplotlib (`uv run --with matplotlib python`).
4. Write two sentences an operator could act on.

Reuse `RULE_SQL` and `rule_pandas`: only the raw condition changes. Try it before opening the answer.

Answer

```
DELTA_C = 5 * 5 / 9  # 5 degF of *difference* -> 2.78 degC
ZERO_F_IN_C = lab.f_to_c(0.0)  # dropout value in the SI history
CONFIRM_SAT = 6  # 6 rows x 5 min = 30 minutes
p = {"delta": DELTA_C, "z": ZERO_F_IN_C, "confirm": CONFIRM_SAT}

SAT_COND = "fan_status = 1 AND abs(sat - sat_sp) > :delta"
SAT_NODROP_COND = SAT_COND + " AND abs(sat - :z) > 1e-6"  # item 1: a
        dropout row is not a fault
sat_sql = lab.query(conn, RULE_SQL.replace("__RAW__", SAT_COND), p)
sat_nd_sql = lab.query(conn, RULE_SQL.replace("__RAW__",
        SAT_NODROP_COND), p)

off = (h.sat - h.sat_sp).abs() > DELTA_C
dropout = np.isclose(h.sat, ZERO_F_IN_C)
sat_pd = rule_pandas(h, (h.fan_status == 1) & off, CONFIRM_SAT)
sat_nd_pd = rule_pandas(h, (h.fan_status == 1) & off & ~dropout,
        CONFIRM_SAT)
lab.assert_parity(sat_pd, sat_sql, keys=["equipment_id", "ts"],
        label="SAT rule")
lab.assert_parity(
    sat_nd_pd, sat_nd_sql, keys=["equipment_id", "ts"], label="SAT
        rule, dropouts excluded"
)

# item 1: what did excluding dropouts change?
n_drop = int(((h.fan_status == 1) & dropout).sum())
print(
    "running rows with sat dropout:",
    n_drop,
    "| raw fault rows",
    int(sat_pd.raw_fault.sum()),
    "->",
    int(sat_nd_pd.raw_fault.sum()),
    "| confirmed rows",
    int(sat_pd.fault.sum()),
    "->",
    int(sat_nd_pd.fault.sum()),
)

# item 2: hours per AHU per month (the confirmed fault rows, dropouts
        excluded)
lab.to_sqlite(sat_nd_sql, conn, "lab_sat", index=["equipment_id",
        "ts"])
SAT_MONTHLY_SQL = MONTHLY_SQL.replace("lab_fc1",
        "lab_sat").replace("fc1_hours", "sat_hours")
sat_monthly_sql = lab.query(conn, SAT_MONTHLY_SQL)
when = pd.to_datetime(sat_nd_pd.ts, unit="s", utc=True)
sat_monthly_pd = (
    sat_nd_pd.assign(month=when.dt.strftime("%Y-%m"))
    .groupby(["equipment_id", "month"])["fault"]
    .sum()
    .mul(5 / 60)
    .rename("sat_hours")
    .reset_index()
)
lab.assert_parity(sat_monthly_pd, sat_monthly_sql, label="SAT hours per
        month")
```

```
print(sat_monthly_sql.pivot(index="month", columns="equipment_id",
        values="sat_hours").round(1))

# item 3: the worst day per AHU (hours of confirmed fault), both ways
DAILY_SQL = """
SELECT equipment_id, strftime('%Y-%m-%d', ts, 'unixepoch') AS day,
        sum(fault) * 5 / 60.0 AS sat_hours
FROM lab_sat GROUP BY equipment_id, day
"""
daily_sql = lab.query(conn, DAILY_SQL)
day = when.dt.strftime("%Y-%m-%d")
daily_pd = (
    sat_nd_pd.assign(day=day)
    .groupby(["equipment_id", "day"])["fault"]
    .sum()
    .mul(5 / 60)
    .rename("sat_hours")
    .reset_index()
)
lab.assert_parity(daily_pd, daily_sql, label="SAT hours per day")
worst = (
    daily_sql.sort_values(["sat_hours", "day"], ascending=[False,
        True])
    .groupby("equipment_id")
    .head(1)
)
print(worst.to_string(index=False))
assert n_drop == 14 and int(sat_pd.raw_fault.sum()) ==
        int(sat_nd_pd.raw_fault.sum()) == 10007
w = worst.set_index("equipment_id")
assert w.loc["AHU_1", "day"] == "2026-05-18" and abs(w.loc["AHU_1",
        "sat_hours"] - 24.0) < 1e-9
assert sat_monthly_sql.groupby("equipment_id").sat_hours.sum().round(1).tolist()
        == [416.5, 298.0]


# no-run
# item 3, the chart (needs matplotlib: uv run --with matplotlib python)
import matplotlib.pyplot as plt

d = worst.iloc[0]
one = h[
    (h.equipment_id == d.equipment_id)
    & (pd.to_datetime(h.ts, unit="s", utc=True).dt.strftime("%Y-%m-%d")
        == d.day)
]
t = pd.to_datetime(one.ts, unit="s", utc=True)
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(t, one.sat, label="sat (degC)")
ax.plot(t, one.sat_sp, label="sat_sp (degC)")
ax.fill_between(
    t, one.sat.min(), one.sat.max(), where=one.fan_status == 1,
        alpha=0.1, label="fan on"
)
ax.legend()
plt.show()
```

1. **Dropouts.** 14 running rows have `sat` at exactly 0 °F. Excluding them removes **0** raw fault rows (10,007 before and after). Why? These are whole-row dropouts: `sat_sp`, `mat` and `rat` read 0 °F too, so `abs(sat - sat_sp)` is 0 and the rule never fired on them anyway. It's still the right habit: a rule comparing `sat` against a *different* reference (like `mat`) would have fired on every one of them. Note the unit trap again: a dropout is -17.78 °C in this history, never 0.
2. **Hours.** AHU_1 has about 417 confirmed hours and AHU_2 about 298, mostly in May and June. That's far more than FC1 found (114 h and 1.5 h).
3. **Worst day.** AHU_1 on 2026-05-18 is off its setpoint by more than 2.8 °C for all 24 hours; AHU_2's worst is 2026-06-16 at 23.1 hours. Whole days like that are a stuck valve, damper or setpoint, not weather.
4. Something like: "AHU_1's discharge air stayed more than 2.8 °C from setpoint for the whole of 18 May and about 417 hours since March. Check the cooling valve and the discharge sensor on AHU_1, and look at what changed in May." Be specific, give the date and a first thing to check.

Because Day 14's rule and this one share `RULE_SQL` and `rule_pandas`, adding a third rule is one new condition string and one new boolean mask, with parity checking the pair.

Next: **Unit 2** builds a tiny triple store in Python, so the Turtle above stops being mysterious.

Reading — Week 3

The lessons first, then these for exact behavior.

- SQLite window functions
- pandas: windowing operations
- SQLite query planner
- SQLite EXPLAIN QUERY PLAN
- Open-FDD SQL cookbook
- TimescaleDB hypertables
- TimescaleDB continuous aggregates